

Universidad Tecnológica Nacional
Tecnatura Universitaria en Programación a Distancia

Programación III

Trabajo Práctico Integrador

Food Store

Sistema de gestión de pedidos de comida
Frontend web (TypeScript + Vite) y backend JPA / Hibernate / H2

Patricio Sussini
Programación III

1. Introducción

Este informe acompaña la entrega del Trabajo Práctico Integrador de Programación III. El proyecto, Food Store, es un sistema de gestión de pedidos de comida resuelto en dos partes independientes y complementarias. La Parte 1 es un frontend web hecho con TypeScript y Vite que consume datos desde archivos .json locales mediante `fetch()`, simulando la futura API REST. La Parte 2 es un backend de consola en Java con JPA, Hibernate y base de datos H2 en archivo, que implementa la capa de persistencia y la lógica de negocio sobre las entidades Categoría, Producto, Usuario y Pedido con sus líneas de detalle. A continuación resumo el modelo, las decisiones técnicas que me parecieron más importantes y cómo se ejecuta cada parte.

2. Backend JPA / Hibernate

2.1. Modelo de dominio

El modelo parte de una clase abstracta **Base** (`@MappedSuperclass`) con los campos comunes `id`, `eliminado` y `createdAt`. De ella extienden cinco entidades: `Categoría`, `Producto`, `Usuario`, `Pedido` y `DetallePedido`. `Pedido` implementa además la interfaz `Calculable` (`calcularTotal()`). Los valores cerrados se modelan con enums: `Rol` (`ADMIN`, `USUARIO`), `EstadoPedido` (`PENDIENTE`, `CONFIRMADO`, `TERMINADO`, `CANCELADO`) y `FormaPago` (`TARJETA`, `TRANSFERENCIA`, `EFFECTIVO`).

Las relaciones son unidireccionales desde el lado “uno”: `Producto` conoce su `Categoría` (`@ManyToOne`); `Pedido` posee sus `DetallePedido` (`@OneToMany` con `cascade=ALL`, una composición); cada `DetallePedido` referencia al `Producto`; y `Usuario` posee su colección de `Pedidos`. Las bajas son siempre lógicas (`eliminado = true`): el registro queda en la base pero no aparece en los listados activos.

2.2. Repositorios y consultas JPQL

Un **BaseRepository<T>** genérico concentra el CRUD común (`guardar`, `buscarPorId`, `listarActivos`, `eliminarLogico`). Cada método abre su propio `EntityManager` y lo cierra en un bloque `finally`; las escrituras manejan la transacción con `begin/commit` y `rollback` ante excepción. `guardar()` usa `persist()` cuando el `id` es `null` (alta) o `merge()` cuando ya existe (actualización), y `buscarPorId()` devuelve un `Optional`. Sobre esa base, cada repositorio agrega sus consultas JPQL tipadas con parámetros nombrados: `CategoriaRepository` con `buscarProductosPorCategoria`; `UsuarioRepository` con `buscarPorMail` (que valida unicidad y devuelve `Optional`) y `buscarPedidosPorUsuario`; y `PedidoRepository` con `buscarPorEstado`, reutilizado por el reporte de total facturado.

2.3. Menú de consola

La clase `Main` presenta un menú principal con cinco secciones —Categorías, Productos, Usuarios, Pedidos y Reportes— más la salida, que cierra el `EntityManagerFactory`. Cada entidad tiene su ABM completo con validaciones (nombre obligatorio, precio mayor a cero, stock no negativo, mail único) y el criterio de que, al modificar, un campo vacío conserva el valor anterior. Los reportes resuelven productos por categoría, pedidos por usuario, pedidos por estado y total facturado, este último sumando los pedidos en estado `TERMINADO` y formateando el resultado a dos decimales.

2.4. El alta de pedido

Es la operación más compleja y la resolví como una única transacción atómica. Primero valido todo en memoria (usuario activo, forma de pago, y por cada ítem: producto existente, disponible y con stock suficiente), guardando sólo el id del producto y la cantidad en una lista temporal. Recién al confirmar abro un único EntityManager y una sola transacción: recupero cada Producto con `em.find()` —queda gestionado, así que el descuento de stock se sincroniza solo—, armo los `DetallePedido` con `pedido.addDetallePedido(cantidad, producto)`, calculo el total y persisto el Pedido, cuyo `cascade=ALL` arrastra los detalles. Si algo falla, se hace rollback completo y la base no se modifica.

3. Frontend web

El frontend está hecho con TypeScript y Vite, sin frameworks, como aplicación multi-página (una entrada HTML por pantalla) con estilos propios en CSS. Respeta la estructura de la consigna (`src/pages/{auth, store, client, admin}`), con los tipos separados por entidad y las utilidades en `src/utils/`.

El login compara credenciales contra `usuarios.json`; si coinciden, guarda el usuario en `localStorage` (sin la contraseña) y redirige según el rol —admin al panel, usuario a la tienda—. El cliente cuenta con catálogo (filtro por categoría, búsqueda en tiempo real y ordenamiento, todo del lado del cliente), detalle con validación de stock, carrito persistido en `localStorage`, checkout con forma de pago e historial filtrado por usuario. El panel de administración suma un dashboard con estadísticas y el ABM de categorías, productos y pedidos con cambio de estado.

Dos decisiones que vale la pena aclarar. El costo de envío es una constante fija (`ENVIO = 500`) y el total del pedido es subtotal más envío. Y toda lectura de datos pasa por una capa de fetch aislada (`src/utils/api.ts`): para conectar el backend de la Parte 2 alcanza con cambiar las rutas de los endpoints (por ejemplo `/data/productos.json` por `/api/products`) sin tocar el resto de la aplicación. Las escrituras del panel admin no modifican los `.json` reales: se apoyan en un overlay sobre `localStorage` para que la demostración sea usable, manteniendo el punto central de la consigna de no persistir en los archivos de datos.

4. Ejecución y entrega

El backend (carpeta `backend/`) es un proyecto Gradle con Java 21 que se ejecuta con `./gradlew run`; la base H2 se crea sola en `./data/jpa_db.mv.db` al primer arranque y los datos se cargan desde el menú en orden: categorías, productos, usuarios y pedidos. El frontend (carpeta `frontend/`) se ejecuta con `npm install && npm run dev` en `http://localhost:5173` e incluye datos de prueba en `public/data/` (cinco categorías, diecinueve productos —uno marcado como no disponible—, tres usuarios y cuatro pedidos con los cuatro estados posibles).

Credenciales de prueba del frontend:

Rol	Mail	Contraseña
ADMIN	admin@admin.com	123456
USUARIO	cliente@food.com	cliente123

USUARIO

maria@food.com

maria123

Ambos proyectos se entregan comprimidos en un único .zip con las carpetas frontend/ y backend/, cada una con su propio README.md. La entrega se completa con el video explicativo, cuyo enlace se deja en los comentarios del aula virtual.